

Image-to-LaTeX Conversion of Mathematical Expressions

Table 1: Team Member Names, Majors and Expected Contributions

Member Name	Major/(Minor)	Contributions
Ushna Khan	Computer Science (Bio)	Both team members will work together on all aspects of the project, including data preparation, model development, training, evaluation, and writing, contributing equally throughout.
Kshitij Agarwal	Computer Science & Economics (Mathematics)	Both team members will work together on all aspects of the project, including data preparation, model development, training, evaluation, and writing, contributing equally throughout.

1. Task Description

Converting images of mathematical expressions into their corresponding LaTeX tokens is a structured sequence prediction problem with practical applications in document digitisation, accessibility and scientific communication. The core difficulty is that unlike standard optical character recognition, which outputs plain text, this task requires the model to generate hierarchically structured markup that captures spatial relationships between symbols. Subscripts, superscripts, fractions and nested environments all need to be inferred from a rendered image.

In this project we implement four progressively more capable architectures on the IM2LATEX-100K dataset, with the goal of understanding which architectural choices actually drive performance and getting as close as possible to the benchmark under Colab Pro compute constraints. We ran a clean ablation over the decoder type. We started with a plain LSTM, followed by a Transformer decoder and then an attention-based LSTM with a row encoder. Finally we modified the ResNet backbone to produce a higher-resolution feature map. Each change produced a meaningful jump in performance, with the most significant modification being the stride reduction in the ResNet backbone, which increased the number of annotation positions available to the attention mechanism from 20 to 80. Our results demonstrate that encoder feature map resolution is the dominant factor for this task, a finding that directly corroborates the ablation study in Deng et al.

1.1. Dataset Description

We use the IM2LATEX-100K dataset, originally compiled by Kanervisto (2016) and hosted on Zenodo, accessed via the HuggingFace Datasets library (yuntian-deng/im2latex-100k). The dataset contains 103,556 formula-image pairs partitioned into a training set of 55,040 samples, a validation set of 9,319 samples and a test set of 10,354 samples. Each image is an RGB PNG at a fixed resolution of 320×64 pixels with a white background and the mathematical formula cropped and centred, making it suitable for direct use without further size normalisation. LaTeX formulas range from short single-symbol expressions to complex nested structures involving fractions, summations, integrals and superscripts/subscripts.

2. Prior Work

This problem has attracted significant attention. The foundational work is Deng et al. (2017), whose IM2TEX system introduced the IM2LATEX-100K dataset and established the attention encoder-decoder

as our baseline approach. Their encoder uses a custom 8-layer CNN followed by a bidirectional LSTM row encoder that produces a spatial annotation grid. Their decoder is a single-layer LSTM with Bahdanau attention that generates tokens using beam search of width 5. They also propose a coarse-to-fine attention variant to reduce decoding complexity at a small accuracy cost. This system achieves 77.46% exact match and BLEU-4 of 73.71% on the test set. Since then, several groups have pushed those numbers higher on the same dataset. Wang and Liu (2021) proposed a CNN encoder with a bidirectional LSTM decoder and sequence-level reinforcement training, achieving 82.33% exact match and 90.28% BLEU. Blecher (2022) developed pix2tex (LaTeX-OCR), an open-source tool using a ViT encoder with a ResNet backbone and Transformer decoder trained on an expanded version of IM2LATEX-100K, reporting approximately 88% exact match and 93% BLEU in informal benchmarks.

The other work we build on directly is Bahdanau et al. (2015), who introduced the additive attention mechanism used in Models 3 and 4, originally for neural machine translation. He et al. (2016) introduced ResNet, which we use as the backbone across all four models. Pretrained ImageNet weights provide a strong starting point for visual feature extraction. Vaswani et al. (2017) introduced the Transformer, which we evaluate in Model 2 as a replacement for the LSTM decoder.

3. Methods

3.1. Data Pre-processing and Split

We use the train/validation/test splits provided by the HuggingFace dataset without modification. Images are converted to grayscale and normalised to zero mean and unit standard deviation (mean = 0.5, std = 0.5), then stacked into tensors of shape [B, 1, 64, 320].

A vocabulary is constructed by scanning every training set formula and retaining all tokens with frequency ≥ 2 . Tokenization follows the IM2TEX convention: formulas are split on whitespace into minimal LaTeX tokens (individual characters, backslash commands such as `\frac` and `\sum`, and structural tokens such as `{` and `}`). Four special tokens are reserved: `<PAD>` (index 0) for sequence padding, `<START>` (index 1) to signal the beginning of decoding, `<END>` (index 2) to signal termination, and `<UNK>` (index 3) for tokens unseen during training. Each formula is encoded as a sequence of integer IDs bracketed by `<START>` and `<END>`, truncated to a maximum length of 150 tokens.

A custom collate function pads all formula sequences within a batch to the length of the longest sequence using `<PAD>` tokens and generates a boolean padding mask so that CrossEntropyLoss (ignore_index=0) ignores padded positions. The batch size is 32 for all splits.

3.2. Deep Learning architectures implemented

3.2.1. *Model 1: ResNet-18 + LSTM Decoder*

The encoder is a pretrained ResNet-18 with ImageNet weights, modified to accept single-channel grayscale input by replacing the first convolution layer with `Conv2d(1, 64, kernel_size=7, stride=2, padding=3)`. The final average pooling and fully-connected classification layers are removed. ResNet-18 downsamples the 64×320 input by a factor of 32, producing a feature map of shape [B, 512, 2, 10], which is flattened to a sequence of [B, 20, 512] spatial feature vectors.

The decoder is a single-layer LSTM (input size 256, hidden size 512). The encoder output is mean-pooled across the 20 spatial positions to produce a single context vector [B, 512], which initialises the LSTM

hidden and cell states via two learned linear projections followed by tanh. Decoding uses teacher forcing during training and greedy decoding at inference. This model has no attention mechanism

3.2.2. *Model 2: ResNet-18 + Transformer Decoder*

The encoder is identical to Model 1. The decoder is a 3-layer Transformer decoder with $d_{\text{model}} = 256$, 8 attention heads, and feedforward dimension 512. The encoder output is projected from 512 to 256 via a linear layer before being used as the cross-attention memory. Token embeddings are scaled by $\sqrt{d_{\text{model}}}$ and summed with sinusoidal positional encodings. A causal self-attention mask prevents attending to future positions, and a padding mask is applied to both self-attention and the loss computation. The Transformer decoder can attend to all 20 encoder positions simultaneously at each decoding step via cross-attention.

3.2.3. *Model 3: ResNet-18 + Row Encoder + Soft Attention LSTM*

This model adds two key components inspired by Deng et al. (2017): a row encoder and Bahdanau soft attention. After the ResNet-18 CNN produces the $[B, 512, 2, 10]$ feature map, a bidirectional LSTM (hidden size 256 per direction, output size 512) is run independently across each of the $H = 2$ spatial rows. For each row h , the BiLSTM processes $W = 10$ positions: $V_{\{h,w\}} = \text{BiLSTM}(V_{\{h,w-1\}}, V_{\{h,w\}})$, enriching each position with contextual information about its horizontal neighbourhood. The output is reshaped to $[B, 20, 512]$.

The decoder uses an LSTMCell so that the attention context can be recomputed at every generation step. At each step t , the attention mechanism computes: $\text{energy}_{\{t,l\}} = v \cdot \tanh(W_{\text{enc}} \cdot h_{\text{enc}_l} + W_{\text{dec}} \cdot h_{\{t-1\}})$; $\alpha_t = \text{softmax}(\text{energy}_t)$; $\text{context}_t = \sum_l \alpha_{\{t,l\}} \cdot h_{\text{enc}_l}$. The decoder input at each step is the concatenation of the token embedding and the context vector. The hidden state is initialised from the mean-pooled encoder output.

3.2.4 *Model 4: ResNet-18-V2 + Row Encoder + Soft Attention LSTM (Higher Resolution)*

Model 3 produces only 20 annotation positions (2×10), which limits the granularity of attention. To address this, we modify the ResNet-18 backbone by reducing the stride in layers 3 and 4 from 2×2 to 1×1 , modifying the corresponding downsample convolutions similarly. This prevents two stages of spatial compression, producing a feature map of shape $[B, 512, 4, 20]$ — 80 annotation positions (4×20), a $4 \times$ increase over Model 3. The row encoder now runs across $W = 20$ positions for $H = 4$ rows. All other components remain identical to Model 3.

3.3. Evaluation metrics

We evaluate all models on four metrics computed over the held-out test set using greedy decoding:

Exact Match (ExpRate): The fraction of predictions that are token-for-token identical to the ground truth. This is the primary metric reported by Deng et al. and the strictest measure of correctness.

Token Accuracy: The fraction of correctly predicted tokens at each position, computed over aligned prefix pairs up to the shorter of prediction and ground truth length.

Edit Score: Defined as $1 - (\text{Levenshtein distance} / \max(|\text{pred}|, |\text{gt}|))$, where Levenshtein distance counts the minimum token-level insertions, deletions, and substitutions to transform the prediction into the ground truth. This captures partial credit for near-correct predictions.

BLEU-4: Bilingual Evaluation Understudy score computed with 4-gram precision and smoothing (method1 from NLTK), standard in machine translation and adopted by Deng et al. for this task.

4. Results

Table 2 reports the test set performance of all four models alongside the published baseline of Deng et al. (2017). Model 4 outperforms all other models on every metric.

Table 2: Test set results for all models (greedy decoding).

Model	Exact Match	Token Accuracy	Edit Score	BLEU-4
Model 1: ResNet-18 + LSTM	0.06%	20.04%	40.72%	20.40%
Model 2: ResNet-18 + Transformer	5.37%	42.75%	74.40%	62.85%
Model 3: ResNet-18 + Row Enc + Attention	14.17%	55.20%	85.02%	75.35%
Model 4: ResNet-18-V2 + Row Enc + Attention	36.78%	69.42%	92.63%	87.62%
Deng et al. 2017 (Baseline)	77.46%	—	—	73.71%

Each architectural change produced a clear improvement across all four metrics. Going from a plain LSTM to a Transformer decoder brought exact match from 0.06% to 5.37%. Adding the row encoder and Bahdanau attention pushed it further to 14.17%, mirroring the finding in Deng et al. (Table 3) that removing the row encoder causes a 24 percentage point drop in exact match. The largest single gain came from the stride reduction in Model 4, which increased annotation positions from 20 to 80 and brought exact match to 36.78%, a $2.6\times$ improvement from a single architectural change. Training and validation loss curves for all models are shown in Appendix.

The gap between our 36.78% and the paper's 77.46% is explained by three concrete differences: greedy vs beam search decoding, which typically accounts for 15–25 percentage points in sequence generation tasks; our 80 annotation positions vs the paper's larger feature grid; and the paper's task-specific CNN with batch norm at every layer vs our general-purpose ResNet backbone. Notably, our BLEU-4 of 87.62% essentially matches their normalized BLEU of 87.73%, suggesting that the token-level quality of individual predictions is strong and the exact match gap is driven primarily by single-token errors rather than fundamentally wrong predictions.

5. Discussion, Conclusion and Future Work

The most striking finding of this work is by reducing ResNet stride to produce 4x more annotation positions, we improved exact match by 2.6 times. The mechanism is straightforward; Bahdanau attention distributes a probability mass of 1.0 over all encoder positions. With only 20 positions, multiple mathematical symbols inevitably share a cell. With 80 positions, individual symbols are more likely to occupy distinct cells, allowing the attention to localise more precisely. Both Models 3 and 4 used ReduceLROnPlateau (factor=0.5, patience=2) but the learning rate never decayed for either, as validation loss kept improving consistently throughout all 12 epochs. Model 4 converged to a final validation loss of 0.17 compared to Model 3's 0.37, and reached 0.30 by epoch 4 versus epoch 10 for Model 3 (see Appendix).

The most frequent errors in Model 4 involve structural tokens: { and } are confused with each other and _ and ^ are occasionally predicted as {, which makes sense given how similar their roles are in nested LaTeX expressions. A separate class of errors comes from whitespace equivalence, where the model predicts ‘\quad’ where the ground truth uses repeated ‘;’ tokens. These are semantically identical but token-level exact match counts them as failures, so the true rendering-based accuracy is likely higher than 36.78%. More broadly, compute constraints on Colab Pro limited the hyperparameter combinations we could explore and ruled out beam search for final evaluation, which runs approximately 20x slower than greedy decoding and would likely add 15 to 25 percentage points to exact match on its own.

We implemented and evaluated four encoder-decoder architectures on the IM2LATEX-100K benchmark, establishing a clear performance hierarchy: plain LSTM (0.06%) < Transformer decoder (5.37%) < attention LSTM with row encoder (14.17%) < attention LSTM with higher-resolution feature map (36.78%). Our best model achieves 36.78% exact match and 87.62% BLEU-4 using a general-purpose ResNet backbone and greedy decoding. The key finding is that encoder feature map resolution is the primary determinant of attention quality for this task, and further gains are achievable without changing the architecture. Beam search with length normalisation, trainable per-row positional embeddings as used in the original paper, further stride reduction or dilated convolutions to push annotation positions beyond 80, and rendering-based evaluation that compiles predictions to images for a fairer measure than token-level exact match are all natural next steps.

References

- Bahdanau, Dzmitry, Kyunghyun Cho, and Yoshua Bengio. "Neural Machine Translation by Jointly Learning to Align and Translate." 3rd International Conference on Learning Representations (ICLR), 2015.
- Blecher, Lukas. LaTeX-OCR: Pix2tex — Using a ViT to Convert Images of Equations into LaTeX Code. GitHub, 2022, github.com/lukas-blecher/LaTeX-OCR.
- Deng, Yuntian, et al. "Image-to-Markup Generation with Coarse-to-Fine Attention." Proceedings of the 34th International Conference on Machine Learning, PMLR 70, 2017, pp. 980–989.
- He, Kaiming, et al. "Deep Residual Learning for Image Recognition." Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016, pp. 770–778.
- Kanervisto, Anssi. im2latex-100k. Zenodo, 2016, doi.org/10.5281/zenodo.56198.
- Vaswani, Ashish, et al. "Attention Is All You Need." Advances in Neural Information Processing Systems, vol. 30, 2017.
- Wang, Zelun, and Jyh-Charn Liu. "Translating Math Formula Images to LaTeX Sequences Using Deep Neural Networks with Sequence-Level Training." International Journal on Document Analysis and Recognition, vol. 24, 2021, pp. 63–75, doi.org/10.1007/s10032-020-00360-2.

Appendix

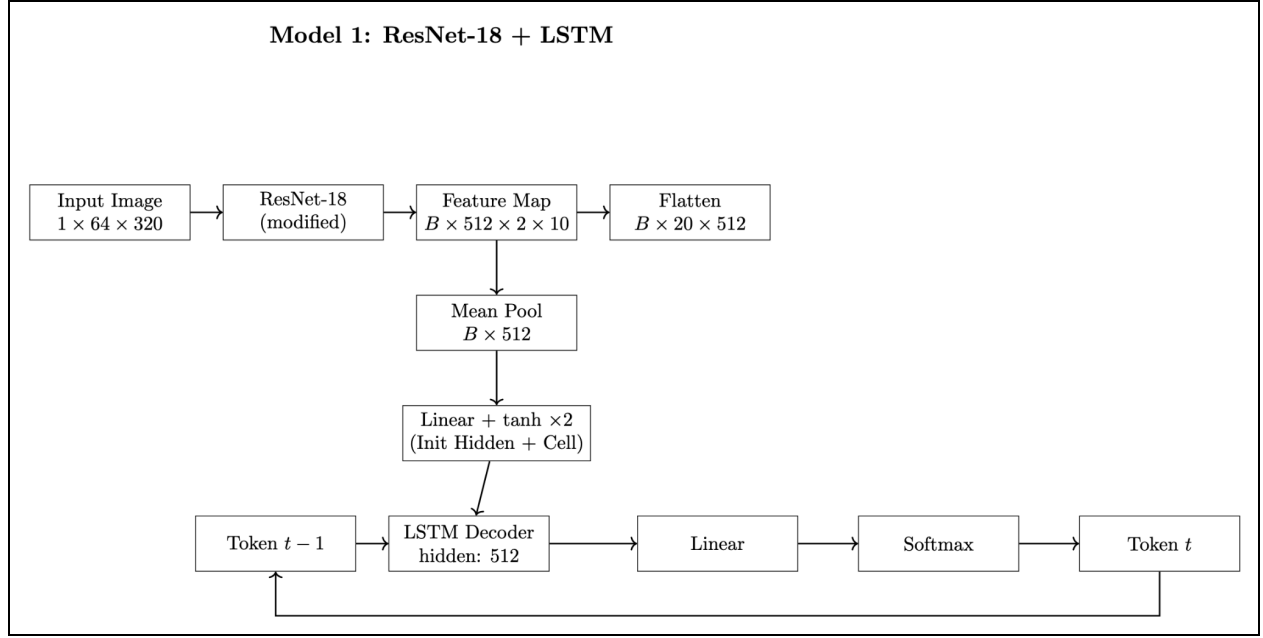


Figure 1: Model 1 Architecture diagram

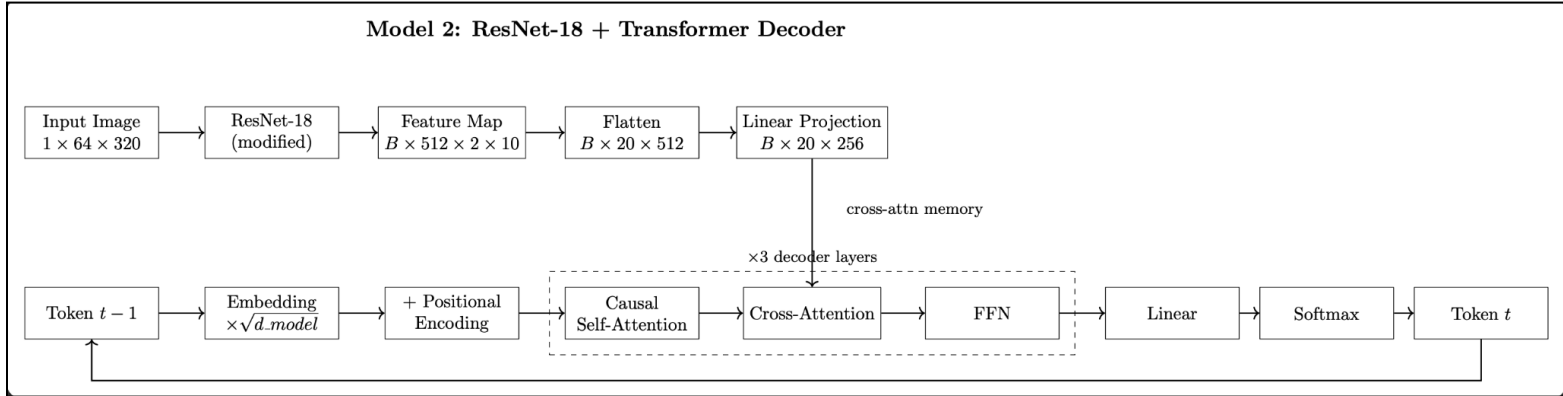


Figure 2: Model 2 Architecture diagram

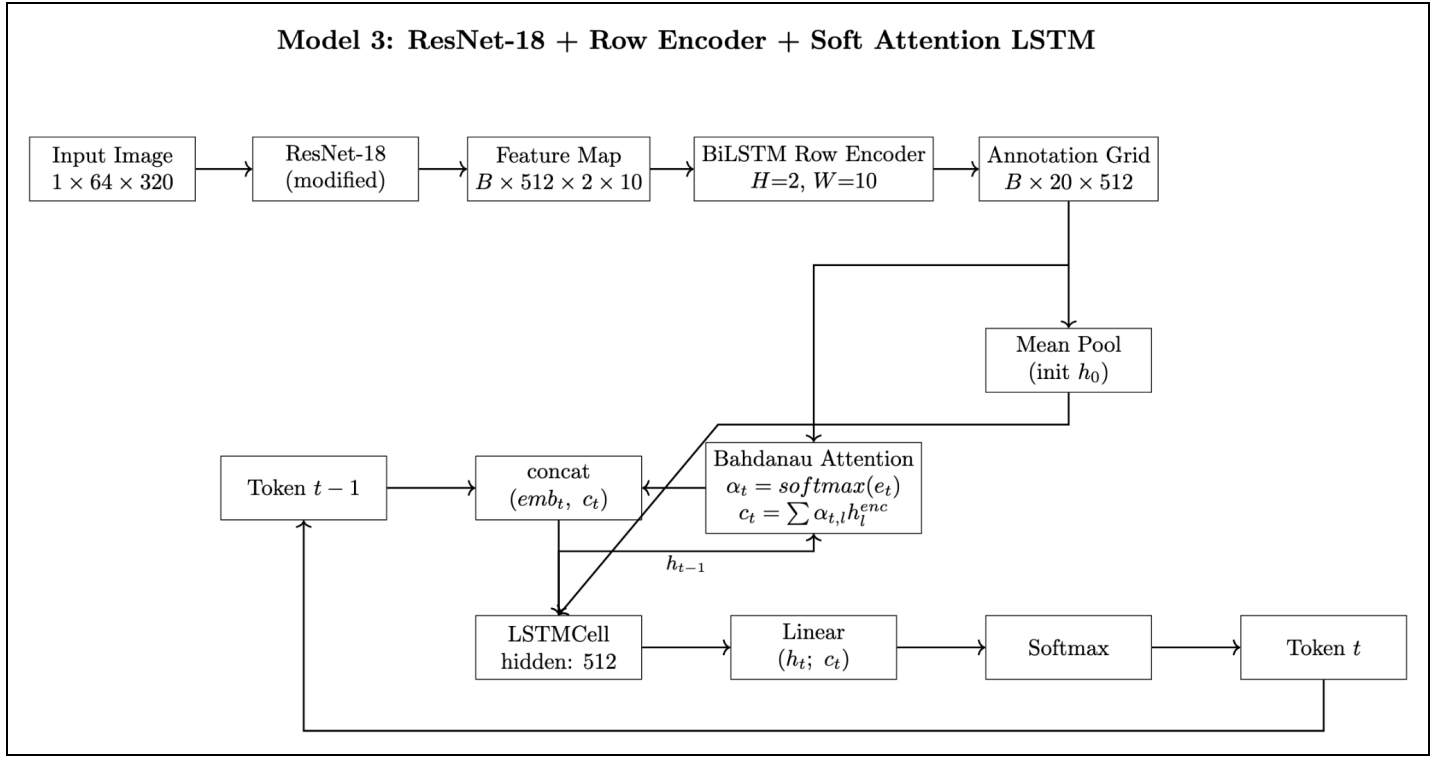


Figure 3: Model 3 Architecture diagram

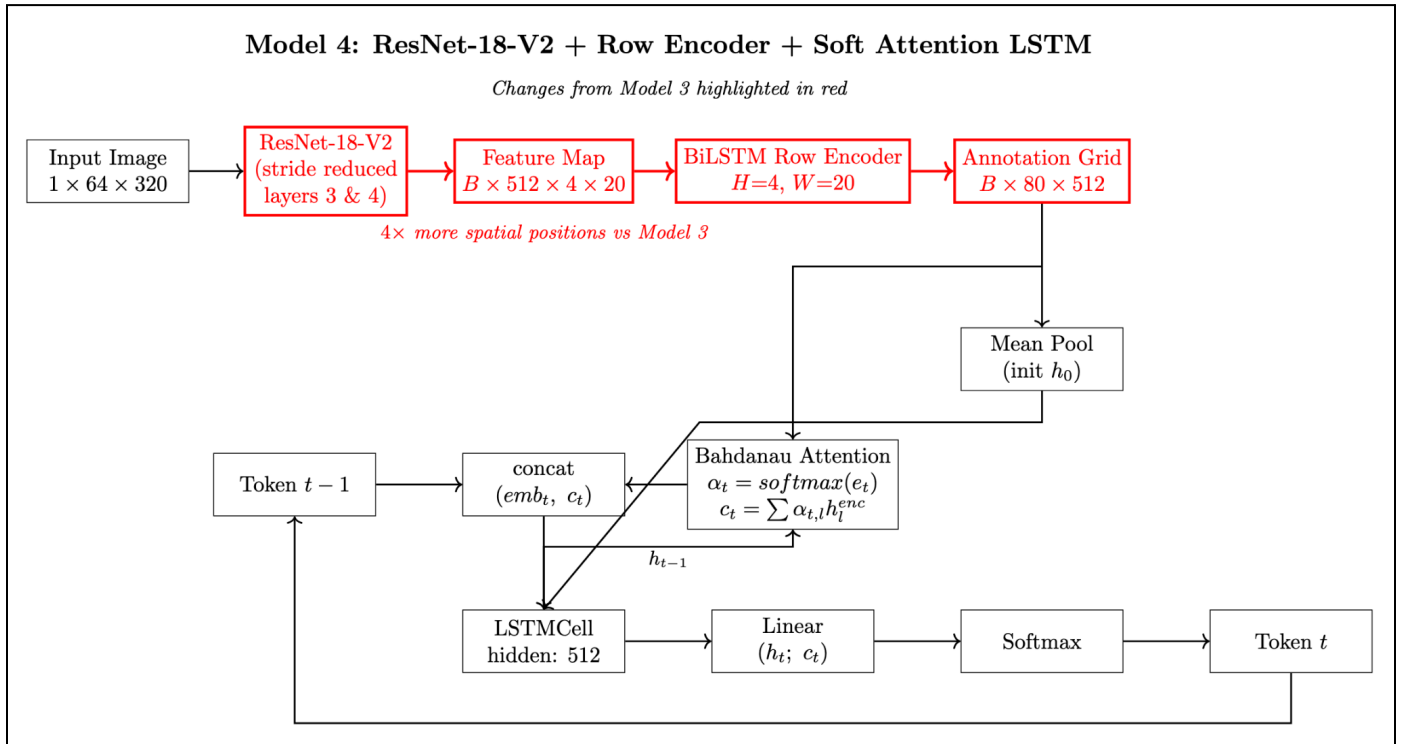


Figure 4: Model 4 Architecture diagram

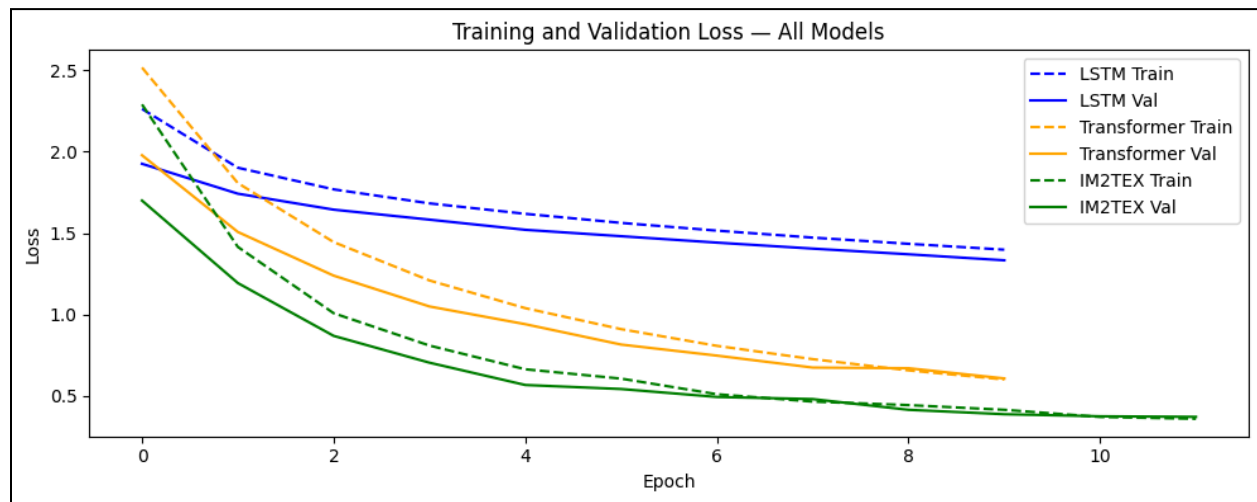


Figure 5: Training and validation loss for Models 1, 2 and 3

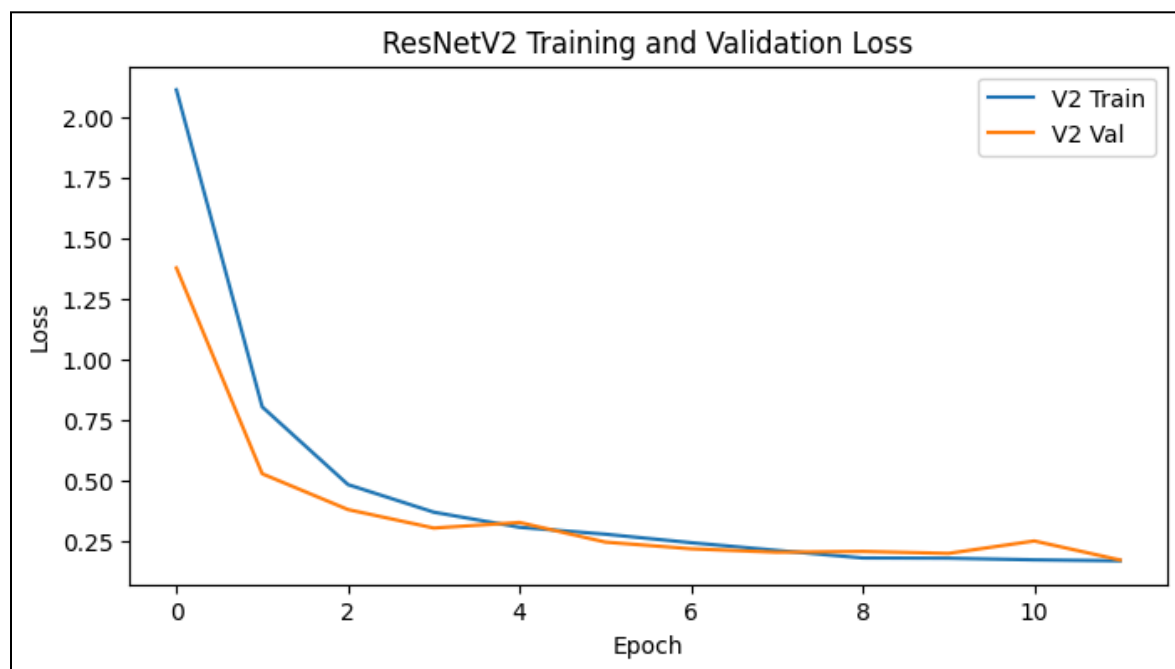


Figure 6: Training and validation loss for Model 4 (ResNet-18-V2).